

CreditCheck – dokumentacja projektu

Opis aplikacji

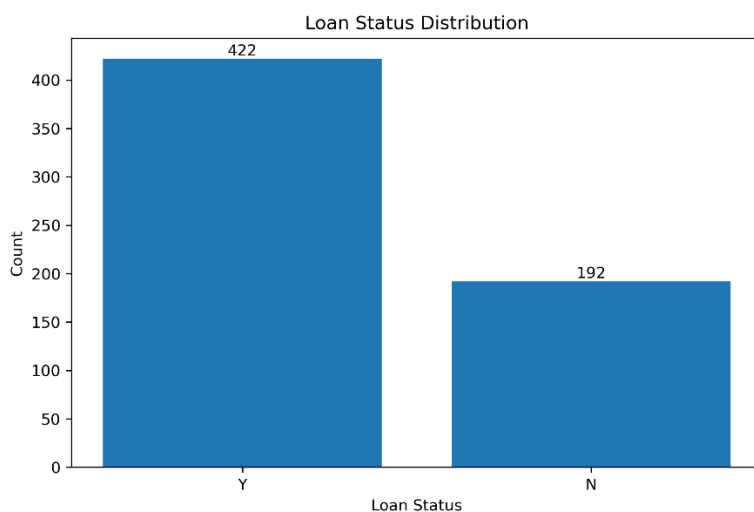
CreditCheck to lokalna aplikacja webowa wspierająca wstępną ocenę szans na uzyskanie kredytu. Użytkownik uzupełnia formularz, a aplikacja przedstawia przewidywaną decyzję modelu, prawdopodobieństwo akceptacji oraz komentarz do podanych danych.

Dataset

W projekcie wykorzystano zbiór Loan Prediction Dataset dostępny na platformie Kaggle. Zbiór zawiera 614 rekordów, 12 cech opisujących wnioskodawcę i parametry kredytu oraz kolumnę docelową Loan_Status określającą decyzję kredytową.

Dane obejmują informacje dotyczące:

- płci wnioskodawcy,
- stanu cywilnego,
- liczby osób na utrzymaniu,
- wykształcenia,
- formy zatrudnienia,
- dochodu wnioskodawcy,
- dochodu współwnioskodawcy,
- kwoty kredytu,
- okresu kredytowania,
- historii kredytowej,
- lokalizacji nieruchomości.



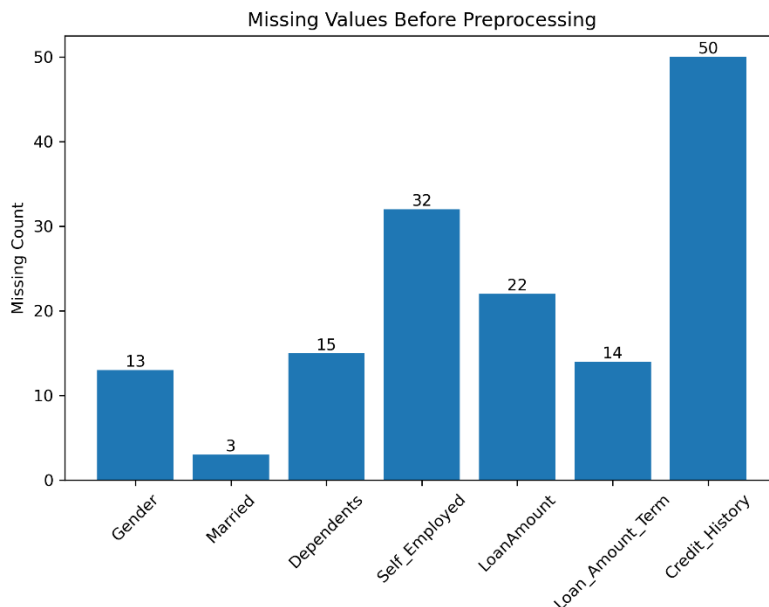
Rysunek 1. Rozkład decyzji kredytowych w zbiorze danych.

Preprocessing danych

Przed treningiem modeli przeprowadzono preprocessing danych w celu poprawy jakości zbioru i przygotowania go do uczenia maszynowego.

Wykonane operacje:

- usunięcie kolumny Loan_ID, która pełniła wyłącznie rolę identyfikatora rekordu,
- uzupełnienie brakujących wartości w danych kategorycznych za pomocą najczęściej występującej wartości (moda),
- uzupełnienie brakujących wartości w danych numerycznych za pomocą mediany,
- zakodowanie zmiennych kategorycznych przy użyciu LabelEncoder,
- utworzenie nowych cech opisujących sytuację finansową wnioskodawcy.

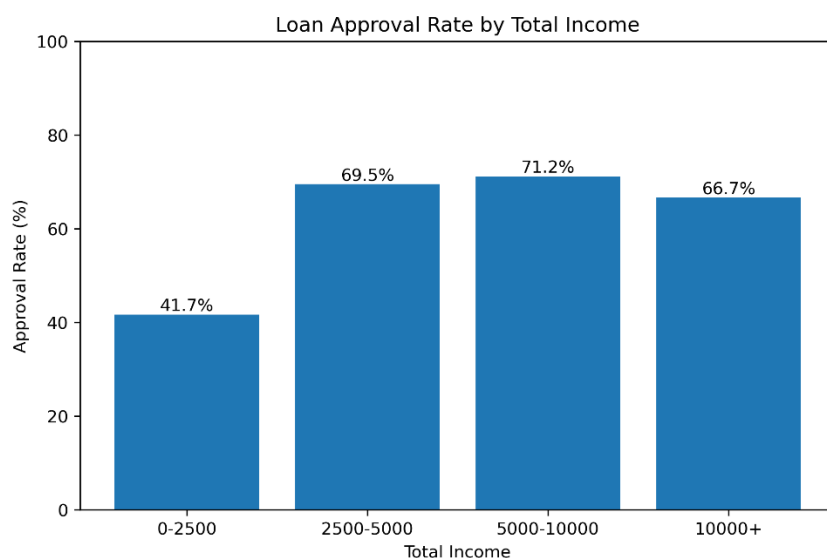


Rysunek 2. Liczba brakujących wartości przed przeprowadzeniem preprocessingu.

W ramach feature engineering dodano:

- $\text{TotalIncome} = \text{ApplicantIncome} + \text{CoapplicantIncome}$,
- $\text{IncomeToLoanRatio} = \text{TotalIncome} / \text{LoanAmount}$.

Dodane cechy pozwoliły modelowi lepiej uwzględnić relację pomiędzy dochodami klienta a wysokością wnioskowanego kredytu.



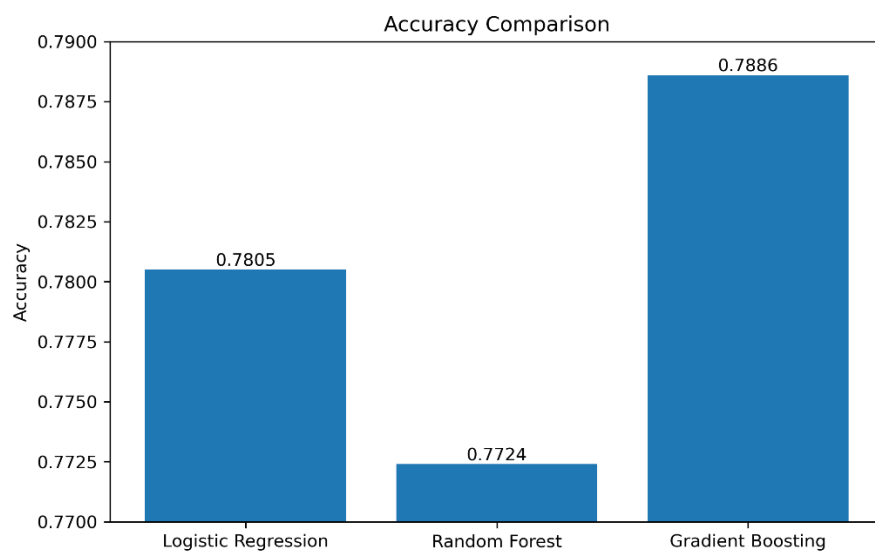
Rysunek 3. Wpływ całkowitego dochodu gospodarstwa domowego na odsetek zaakceptowanych wniosków kredytowych.

Model

W celu wyznaczenia najlepszego rozwiązania porównano trzy modele uczenia maszynowego:

- Logistic Regression,
- Random Forest Classifier,
- Gradient Boosting Classifier.

Modele oceniono przy użyciu metryk Accuracy, Precision, Recall oraz F1 Score.



Rysunek 4. Porównanie skuteczności testowanych modeli klasyfikacyjnych.

Uzyskane wyniki:

Logistic Regression

- Accuracy: 0.7805
- Precision: 0.7573
- Recall: 0.9750
- F1 Score: 0.8525

Random Forest

- Accuracy: 0.7724
- Precision: 0.7708
- Recall: 0.9250
- F1 Score: 0.8409

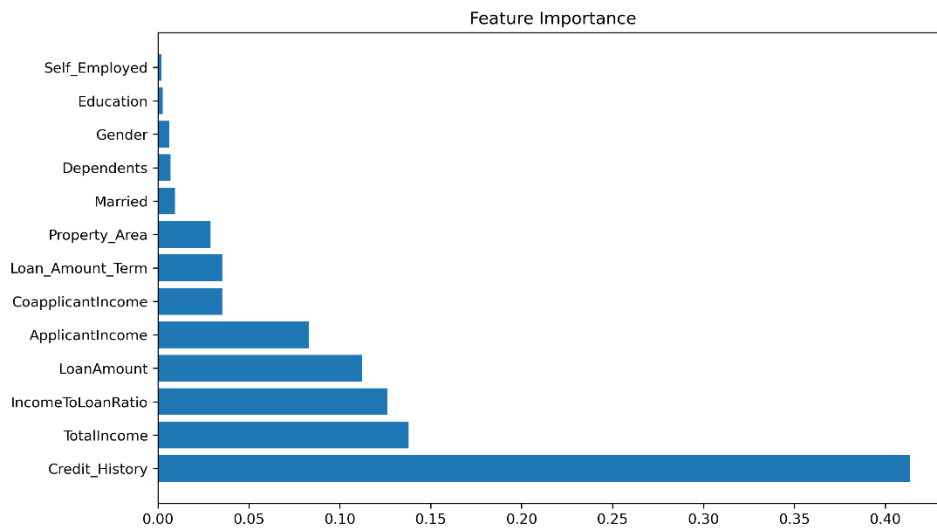
Gradient Boosting

- Accuracy: 0.7886
- Precision: 0.7812
- Recall: 0.9375
- F1 Score: 0.8523

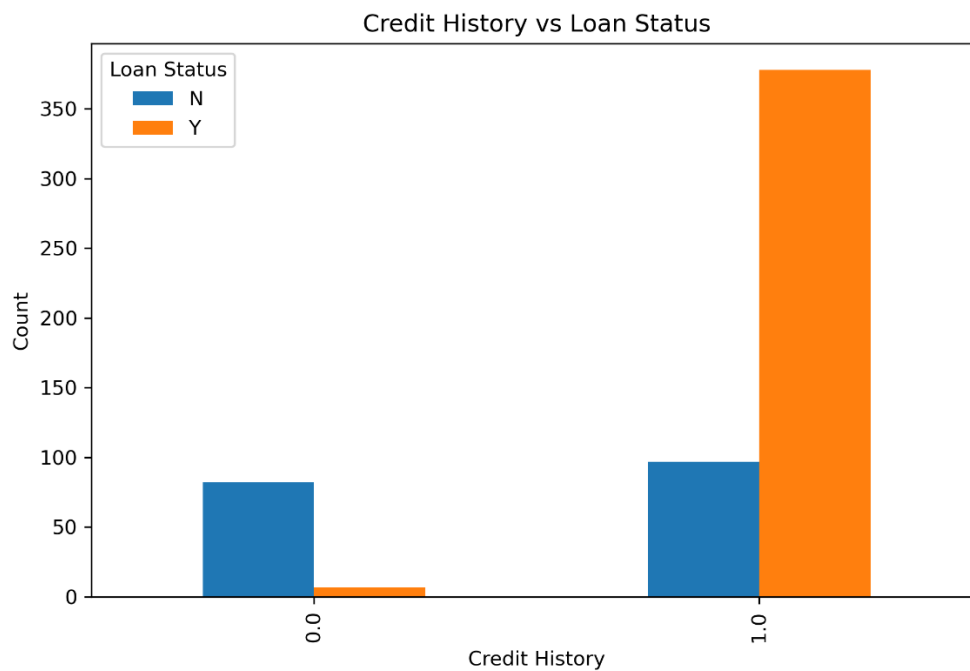
Model Gradient Boosting został wybrany ze względu na najwyższą skuteczność spośród testowanych modeli oraz dobrą równowagę pomiędzy Accuracy, Precision, Recall i F1 Score, został zapisany do pliku `credit_model.pkl` i wykorzystany przez aplikację.

Największy wpływ na decyzję modelu miały cechy:

- Credit_History,
- TotalIncome,
- IncomeToLoanRatio,
- LoanAmount.



Rysunek 5. Znaczenie poszczególnych cech w modelu Gradient Boosting.



Rysunek 6. Zależność pomiędzy historią kredytową a decyzją kredytową.

Wektor wejściowy

Model wykorzystuje 13 cech wejściowych:

- Gender,
- Married,
- Dependents,
- Education,

- Self_Employed,
- ApplicantIncome,
- CoapplicantIncome,
- LoanAmount,
- Loan_Amount_Term,
- Credit_History,
- Property_Area,
- TotalIncome,
- IncomeToLoanRatio.

Dwie ostatnie cechy są generowane podczas preprocessingu danych i nie występują bezpośrednio w oryginalnym zbiorze danych.

Wektor wyjściowy

Model zwraca binarną decyzję kredytową:

- 0 – odrzucenie wniosku kredytowego,
- 1 – akceptacja wniosku kredytowego.

Dodatkowo model umożliwia wyznaczenie prawdopodobieństwa przyznania kredytu przy użyciu metody `predict_proba()`, co pozwala na prezentację bardziej szczegółowej informacji o wyniku predykcji.

Uruchomienie

1. Pobierz lub sklonuj repozytorium.
2. Uruchom plik `run.bat`.
3. Przy pierwszym uruchomieniu skrypt:
 - sprawdzi wersję Pythona,
 - utworzy lokalne środowisko `.venv`,
 - zainstaluje wersje bibliotek z `requirements.txt`,
 - uruchomi aplikację w przeglądarce
4. Aplikacja otworzy się w przeglądarce pod adresem `http://localhost:8501`.

Interfejs

Aplikacja została przygotowana jako lokalna aplikacja internetowa z wykorzystaniem biblioteki Streamlit. Interfejs składa się z formularza podzielonego na sekcję danych wnioskodawcy oraz sekcję danych kredytu.

Użytkownik podaje między innymi płeć, stan cywilny, dochód, liczbę osób na utrzymaniu, formę zatrudnienia, obecność i dochód współwnioskodawcy, kwotę kredytu, okres kredytowania, historię spłaty zobowiązań oraz lokalizację nieruchomości.

Po wybraniu przycisku „Sprawdź decyzję kredytową” aplikacja przekazuje przygotowane dane do modelu. Następnie prezentuje przewidywaną decyzję, prawdopodobieństwo akceptacji oraz komentarze dotyczące podanych informacji.

System testowy

Aplikację przetestowano lokalnie na:

- Windows 11,
- Python 3.13,
- Streamlit 1.41.1,
- scikit-learn 1.6.1.

Aplikacja była testowana na przeglądarkach Chrome/OperaGX.

Walidacja i obsługa błędów

Przed wykonaniem predykcji aplikacja sprawdza poprawność podstawowych danych formularza. Dochód wnioskodawcy oraz wnioskowana kwota kredytu muszą być większe od zera. Jeżeli użytkownik zadeklaruje współwnioskodawcę, wymagane jest podanie jego dochodu.

Aplikacja sprawdza również dostępność i zgodność modelu. Obsługiwane są następujące sytuacje:

- brak pliku modelu,
- uszkodzony plik modelu,
- model wykorzystujący inny zestaw lub kolejność cech,
- plik, który nie zawiera modelu predykcyjnego,
- błąd podczas wykonywania predykcji.

Architektura projektu

Projekt został podzielony na osobne moduły zgodnie z ich odpowiedzialnością:

- data – dane surowe i dane przygotowane do treningu,
- model – preprocessing, trening, porównanie modeli oraz zapis modelu do pliku .pkl,
- app – aplikacja Streamlit, formularz, walidacja danych i prezentacja wyniku,
- tests – testy integracji aplikacji z zapisanym modelem.

Taki podział oddziela przygotowanie danych, trenowanie modelu i działanie aplikacji webowej.

Testowanie aplikacji

W projekcie przygotowano 11 testów integracyjnych. Sprawdzają one:

- zgodność danych formularza z wektorem wejściowym modelu,
- kodowanie danych tekstowych,
- przekazywanie płci wybranej przez użytkownika,
- walidację dochodu i kwoty kredytu,
- obsługę dochodu współwnioskodawcy,
- działanie predykcji z rzeczywistym modelem .pkl,

- obsługę braku i uszkodzenia modelu,
- wykrywanie niezgodnego zestawu cech,
- obsługę błędu predykcji,
- brak fikcyjnego wyniku w przypadku błędu.

Testy można uruchomić poleceniem:

```
..\venv\Scripts\python.exe -m unittest discover -s tests -v
```

Podczas ostatniej weryfikacji wszystkie 11 testów zakończyło się powodzeniem.

Jakość kodu

Do kontroli jakości kodu wykorzystano narzędzie Pylint. Sprawdzenie można uruchomić poleceniem:

```
..\venv\Scripts\python.exe -m pylint app\streamlit_app.py model\train_model.py
tests\test_model_integration.py
```

Podczas ostatniej weryfikacji projekt uzyskał wynik 9,82/10.

Podział pracy

Piotr Balcerzak

- przygotowanie i analiza datasetu,
- preprocessing danych,
- trenowanie modeli uczenia maszynowego,
- porównanie skuteczności modeli,
- wybór najlepszego modelu,
- zapis modelu do pliku .pkl.

Patrycja Ługiewicz

- utworzenie i konfiguracja repozytorium GitHub,
- przygotowanie struktury projektu,
- implementacja aplikacji webowej w Streamlit,
- stworzenie formularza użytkownika,
- integracja aplikacji z modelem,
- przygotowanie README,
- przygotowanie dokumentacji,
- przygotowanie prezentacji projektu.